

Machete con pasos para usar HUnit

1. Crear los archivos

- Un archivo para las funciones.
- Otro archivo para los tests.

2. Definir el módulo de funciones a exportar

Arriba de todo del archivo de funciones, hay que poner:

```
module NombreDeseado where
```

Esto define el nombre del módulo, que luego se usará en el archivo de tests para importar las funciones. El nombre del módulo tiene que coincidir con el nombre del archivo que contiene las funciones. Por ejemplo, si el archivo se llama `MisFunciones.hs`, hay que escribir:

```
module MisFunciones where
```

3. Definir el código de las funciones

En el archivo de las funciones, escribir todas las funciones que queremos testear debajo de la declaración del módulo.

4. Importar HUnit y el módulo de funciones en el archivo de tests

Arriba de todo del archivo de tests hay que escribir:

```
import Test.HUnit -- Importamos la biblioteca para los tests
import MisFunciones -- Importamos el módulo que contiene las funciones a testear
```

5. Definir nuestro test suite

Para definir un test suite, lo hacemos como:

```
misTests = test [
    "Nombre del test 1" ~: (llamado a función) ~= valorEsperado1,
    "Nombre del test 2" ~: (llamado a función) ~= valorEsperado2
]
```

Importante recalcar que lo que estamos haciendo es definir una lista de elementos de tipo `Test`, por lo que siempre el último test no lleva una coma al final.

6. (Opcional) Definir un alias para ejecutar los tests

La manera tradicional de correr los tests es desde GHCi escribiendo: `runTestTT misTests`. Pero podemos definir un alias para no tener que escribir `runTestTT` cada vez que queremos ejecutar un test:

```
nombreAlias = runTestTT misTests
```

Con este alias, podemos ejecutar nuestros tests de dos formas en GHCi: haciendo `nombreAlias` o `runTestTT misTests`, luego de cargar el archivo de los tests en GHCi.

Extra: Ejecutar varias Test Suites

Si tenemos más de una suite de tests y queremos ejecutar todas a la vez, por ejemplo, si a nuestros tests de Fibonacci le agregamos una suite de tests para el factorial:

```
runFactorial = runTestTT testsFactorial

testsFactorial = test [
    "Caso base 1: factorial 0" ~: factorial 0 ~= 1,
    "Caso recursivo 1: factorial 1" ~: factorial 1 ~= 1,
    "Caso recursivo 2: factorial 2" ~: factorial 2 ~= 2
]
```

Para no tener que ejecutar `runFibo` y `runFactorial` por separado, podemos agruparlos primero:

```
todosLosTests = test [testsFibo, testsFactorial]
```

Para ejecutar los tests, primero hay que abrir GHCi en la carpeta del proyecto y cargar el archivo que contiene los tests. Por ejemplo:

```
ghci
ghci> :l TestsDeMisFunciones.hs
```

Una vez cargado el archivo, podemos ejecutar la suite directamente con:

```
runTestTT todosLosTests
```

O podemos definir un alias para la suite:

```
runTodos = runTestTT todosLosTests
```

Después de abrir GHCi y cargar el archivo de tests, basta con escribir `runTodos` para ejecutar todas las suites a la vez.